



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068240218

姓 名： 杨陆续

实验（1）

【实验目的】

1. 深入理解基于 AnythingLLM 的 Agent 开发教学项目的整体架构与工作流程。
2. 掌握通过 API 集成外部大模型（AnythingLLM）的查询功能，实现智能问答代理（Agent）的功能扩展。
3. 实践子进程调用、环境变量管理、API 认证、错误处理等编程技能，并完成代码重构、系统提示词优化及项目文档更新。

【实验内容】

1. **理解项目：**分析现有 Agent 开发教学项目的代码结构，明确 chat_client.py、.env 配置文件及各模块间的交互逻辑。
2. **功能开发：**
 1. 创建 practice04 工作目录，并复制 practice03 的代码作为基础。
 2. 在项目中实现 anythingllm_query 功能。具体包括：
 - 使用 subprocess 模块调用 curl 命令，访问 AnythingLLM 的聊天 API
(http://localhost:3001/api/v1/workspace/{workspace_slug}/chat)。
 - 正确处理请求与响应中的中文编码（如 UTF-8）。
 - 在请求的 message 字段中发送用户查询。
 - 从请求头中使用 API 密钥进行认证。
 - 从项目根目录的 .env 文件中读取 ANYTHINGLLM_API_KEY 和 ANYTHINGLLM_WORKSPACE_SLUG 环境变量。
 - 实现完善的错误处理机制，并在遇到问题时查阅相关技术文档。
 3. 对开发的功能进行单元测试与集成测试，确保其稳定运行。
4. **代码重构与集成：**
 - 修改 chat_client.py 文件，添加新的 anythingllm_query 工具定义及其对应的 function call 实现。
 - 更新 Agent 的可用工具（function）列表，将新工具纳入系统。
 - 更新系统提示词（system prompt），明确当用户提问中包含“文档仓库”、“文件仓库”或“仓库”等关键词时，自动触发 anythingllm_query 工具进行查询。
5. **测试与文档：**
 - 对修改后的完整项目进行端到端测试，验证新功能与原有功能的兼容性与正确性。
 - 更新项目根目录的 README.md 文件，详细说明新增的 AnythingLLM 查询功能的使用方法、配置步骤及注意事项。

1.2 实验过程：

1. **环境与项目初始化：**首先，在开发环境中确认 Python 环境、AnythingLLM 服务 (<http://localhost:3001>) 均正常运行。接着，创建 practice04 目录，并将 practice03 目录下的所有源代码文件复制至此，作为本次实验的代码基础。
2. **功能函数实现：**在项目中创建或修改一个专门的模块（如 tools.py 或 anythingllm_client.py），编写 anythingllm_query(query: str) -> str 函数。该函数的核心流程如下：
 - 读取配置：**使用 python-dotenv 库从 .env 文件加载 ANYTHINGLLM_API_KEY 和 ANYTHINGLLM_WORKSPACE_SLUG。
 - 构造请求：**组装 curl 命令字符串。包含 API 端点 URL（注入 workspace_slug）、Authorization: Bearer {api_key} 请求头、Content-Type: application/json 请求头，以及 JSON 格式的请求体 {"message": query}。特别注意对 query 字符串进行适当的转义，以支持中文和特殊字符。
 - 执行调用：**使用 subprocess.run() 执行 curl 命令，捕获其标准输出、标准错误和返回码。
 - 处理响应：**检查返回码，若成功（通常为 0），则从标准输出中解析 JSON 响应，提取出回答内容；若失败，则记录标准错误信息，抛出清晰的异常或返回错误信息。
3. **Agent 集成：**修改 chat_client.py 主逻辑文件。
 - 工具定义：**在工具定义部分，按照 function calling 的规范，添加 anythingllm_query 工具的 JSON Schema 描述，包括名称、描述、参数（query 字符串）等。
 - 函数映射：**在工具函数映射字典中，将 anythingllm_query 工具名指向步骤 2 实现的函数。
 - 提示词更新：**在系统提示词中，增加明确的指令，例如：“当用户的问题涉及‘文档仓库’、‘文件仓库’、‘仓库’中的内容时，你必须使用 anythingllm_query 工具来获取相关信息以回答用户。”
4. **功能测试：**
 - 单元测试：**直接调用 anythingllm_query 函数，传入不同的问题（包括中文问题），验证其能否成功从 AnythingLLM 服务获取回答。
 - 集成测试：**运行更新后的 chat_client.py，启动 Agent 对话。向 Agent 提问如“请告诉我仓库里有哪些关于人工智能的文档？”或“总结一下仓库中项目 A 的要点”，观察 Agent 是否能正确识别关键词、触发工具调用，并返回来自 AnythingLLM 的合理答复。
5. **文档更新：**在 README.md 中新增“AnythingLLM 查询功能”章节。内容包括：功能简介、环境变量配置说明（如何在 .env 中设置 API Key 和 Workspace Slug）、如何使用（触发关键词示例）、常见问题排查（如 API 连接失败、中文乱码等）。

1.2 实验结果：

你：我在文件仓库上传了什么文件
助手：

```
{
  "id": "tool_call_1",
  "type": "function",
  "function": {
    "name": "anythingllm_query",
    "arguments": {
      "message": "我在文件仓库上传了什么文件"
    }
  }
}
```

}执行工具：anythingllm_query

解析后的参数：{'message': '我在文件仓库上传了什么文件'}

成功加载 .env 文件：d:\tmp\.env

[调试] 执行命令：curl -X POST http://localhost:3001/api/v1/workspace/fd69fe51-36c1-4534-bd15-b3b45dd655d9/chat -H Authorization: Bearer KDHP5S3-P6K4N6C-K98M6HA-56Y56SE -H Content-Type: application/json -d {"message": "我在文件仓库上传了什么文件"}

[调试] 响应：{

```
  "id": "66586c71-54cd-46bd-be57-caf02a3997a1",
  "type": "textResponse",
  "sources": [],
  "close": true,
  "error": null,
  "textResponse": "你上传了一个名为 **\"软件工程24402+24068240218+杨陆续（2）.docx\"** 的文件。该文件目前没有描述信息。如果需要进一步了解其内容，请告诉我！",
  "thoughts": []
}
```

工具执行结果：{'status': 'success', 'message': '查询成功', 'response': '你上传了一个名为 **\"软件工程24402+24068240218+杨陆续（2）.docx\"** 的文件。该文件目前没有描述信息。如果需要进一步了解其内容，请告诉我！'}

【实验总结】

本次实验成功完成了对智能对话 Agent（智能体）的功能扩展，通过创建 practice04 工作目录并基于前期代码，实现了一个能够调用外部大模型知识库（AnythingLLM）的查询工具。具体而言，实验过程包括：开发了使用 subprocess 模块调用 curl 命令、访问特定 API 端点、处理中文编码、利用 .env 文件中的环境变量进行密钥认证的 anythingllm_query 功能函数；随后修改了 chat_client.py 主程序，将新工具的定义和调用逻辑集成到 Agent 的 function call 工具列表中，并更新了系统提示词，使 Agent 能在用户提及“仓库”等关键词时自动触发该工具进行智能检索；最后，对整个系统进行了功能测试，并更新了项目 README.md 文档以说明新功能的用法。整个实验系统地实践了从 API 集成、代码重构、系统提示词优化到测试和文档编写的完整开发流程，达到了增强 Agent 利用外部私有知识库能力的核心目标。